Lecture 24 : Project Estimation Techniques (Contd.)

Introduction

In the dynamic world of software development, accurately estimating project parameters like effort, cost, and time is paramount for successful delivery. This lecture delves into the third major category of project estimation techniques: analytical estimation, specifically focusing on Halstead's Software Science. Unlike empirical methods that rely on educated guesses or heuristic approaches that model relationships mathematically, analytical techniques are grounded in scientific principles and basic assumptions about project parameters. Halstead's Software Science stands out as a powerful analytical tool, particularly effective for estimating software maintenance effort, often outperforming its empirical and heuristic counterparts in this domain. Understanding this technique provides a robust framework for quantifying various aspects of a software project, from its inherent complexity to the effort required for its development and ongoing maintenance.

Core Concepts

Term	Definition	Significance
`Empirical Estimation`	A technique based on making an educated guess about initial project parameters, which are then refined to estimate other factors like effort and cost.	Provides initial, quick estimates but lacks scientific rigor, relying heavily on expert judgment.
`Heuristic Estimation`	Assumes that relationships between different project parameters can be modeled using mathematical expressions, often refined with multipliers or cost drivers.	Offers more structured estimation than empirical methods, exemplified by COCOMO models.
`Analytical Estimation`	Derives results from basic assumptions about project parameters, distinguished by its	Provides a more objective and scientifically grounded method for project estimation, as seen in

Term	Definition	Significance
	scientific basis and structured approach.	Halstead's Software Science.
`Halstead's Software Science`	An analytical estimation technique that uses primitive program parameters (operators and operands) to derive metrics for program length, volume, effort, and development time.	Especially useful for estimating software maintenance effort, offering a quantitative basis for project management.
`Operators (η1, N1)`	`η1` is the count of unique operators, and `N1` is the total count of all operators used in a program.	Fundamental building blocks for Halstead's metrics, representing actions or functions within the code.
`Operands (η2, N2)`	`η2` is the count of unique operands, and `N2` is the total count of all operands (variables and constants) used in a program.	Represent the data or entities upon which operators act, crucial for understanding program structure.
`Program Length (N)`	The total usage of all operators and operands in a program, calculated as `N = N1 + N2`.	A basic measure of program size, reflecting the overall number of symbols used.
`Program Vocabulary (η)`	The total number of unique operators and operands used in a program, calculated as `η = η1 + η2`.	Indicates the diversity of elements used in a program, reflecting its complexity at a fundamental level.
`Program Volume (V)`	A measure of program size that accounts for the programming language used, calculated as `V = N * log2(η)`.	Compensates for language differences, providing a more standardized measure of program size in bits.

Term	Definition	Significance
`Programming Effort (E)`	The mental discriminations required to implement, read, and understand a program, calculated as `E = V^2 / V*`.	Quantifies the cognitive load associated with developing and maintaining software, a key project management metric.

Detailed Analysis

1. Categories of Project Estimation Techniques

Project estimation techniques are broadly categorized into three main types, each with distinct characteristics and applications:
* **Empirical Estimation**: This approach relies on making an initial, educated guess about project parameters. Project managers use their experience and intuition to estimate factors like size, which then inform decisions about effort and cost.
* **Characteristics**: Based on subjective judgment, less formal.
* **Examples**: Expert Judgment Technique, Delphi Technique.
* **Heuristic Estimation**: This technique assumes that the relationships between various project parameters can be effectively modeled using mathematical expressions. These models are often refined using multipliers or cost drivers to achieve more accurate estimates.
* **Characteristics**: Uses mathematical models, allows for refinement.
* **Examples**: COCOMO (Constructive Cost Model) models, including Basic, Intermediate, Complete, and COCOMO 2.
* **Analytical Estimation**: Distinct from the other two, analytical techniques derive required results by starting with basic assumptions about project parameters. Crucially, these techniques possess a scientific basis, making their assumptions and methods more rigorous and less vague.
* **Characteristics**: Scientific basis, derived from fundamental assumptions.
* **Example**: Halstead's Software Science. This method is particularly valuable for estimating software maintenance efforts, often outperforming empirical and heuristic techniques in this specific area. It provides better results and more accurately estimates parameters like effort and cost for maintenance.

2. Halstead's Software Science Overview

Halstead's Software Science is an analytical technique designed to estimate various project parameters. It provides a quantitative framework for understanding software

attributes based on low-level program characteristics.
* **Purpose**: To estimate software size, development effort, and development time. It is especially useful for predicting software maintenance effort.
* **Primitive Program Parameters**: Halstead's model is built upon two fundamental types of elements found in any program:
* **Operators**: Symbols or keywords that perform actions (e.g., `+`, `=`, `if`, `while`).
* **Operands**: Variables or constants that are manipulated by operators (e.g., `a`, `b`, `10`).
* **Derived Expressions**: Using these primitive parameters, Halstead's Software Science allows for the derivation of expressions for:
* Overall program length
* Potential minimum volume
* Actual volume
* Language level
* Effort
* Development time

3. Identifying Operators and Operands

To apply Halstead's Software Science, it's crucial to correctly identify and count operators and operands within a program. The lecture provides specific guidelines:
* **Operators**:
* Assignment (`=`), arithmetic (`+`, `-`, `*`, `/`), and logical operators (`&&`, `||`) are counted as operators.
* A pair of parentheses `()` or a block `begin...end` pair are considered single operators.
* A label used as the target of a `GOTO` statement is an operator.
* Control constructs like `if...then...else...endif` and `while...do` are counted as single operators.
* The statement terminator (e.g., semicolon `;` in C) is a single operator.
* A function name in a function call statement is an operator.
* **Operands**:
* Subroutine declarations and variable declarations comprise operands.
* The arguments of a function call are considered operands.
* **Important Distinctions**:
* The function name in a function definition is not counted as an operator.
* The parameter list of a function in its declaration statement is not considered an operand.
* Variable names appearing in declarations are not considered operands; only variables and constants used with operators in expressions are operands.

* **Examples**:
* Consider the expression `a = &b;`

```
a = &b;
```

Here, `a` and `b` are operands. `=`, `&`, and `;` are operators.
* Consider the function call `func(a,b);`

```
func(a,b);
```

Here, `func`, `,`, and `;` are operators. `a` and `b` are operands.

4. Halstead's Metrics: Length and Vocabulary

Once operators and operands are identified and counted, Halstead's Software Science defines fundamental metrics:
* **`η1`**: Number of unique operators used in the program.
* **`η2`**: Number of unique operands used in the program.
* **`N1`**: Total number of operators used in the program.
* **`N2`**: Total number of operands used in the program.
* **Program Length (N)**: Quantifies the total usage of all operators and operands.
* Formula: `N = N1 + N2`
* **Program Vocabulary (η)**: Represents the number of unique operators and operands used.
* Formula: `η = η1 + η2`

5. Halstead's Metrics: Program Volume

Program length alone can be misleading because it depends on programming style and language. Halstead introduced program volume to address this dependency.
* **Dependency on Language**: The length of a program for the same problem can vary significantly based on the programming language and style used. Different languages might produce different length measures for the same underlying logic.
* **Compensation for Language Effect**: Program volume (V) is designed to compensate for the effect of the programming language used, providing a more standardized measure of program size.
* **Formula**: `V = N * log2(η)`
* `N` is the program length (total operators + total operands).
* `η` is the program vocabulary (unique operators + unique operands).
* **Intuitive Idea**: Intuitively, program volume `V` represents the minimum number of bits required to encode the program. To uniquely represent `η` different

identifiers, at least `log2(η)` bits are needed. Thus, `V` approximates the size of the program in a language-independent manner.

6. Halstead's Metrics: Potential Minimum Volume and Program Level

These metrics further refine the understanding of a program's inherent complexity and the abstraction provided by its language.
* **Potential Minimum Volume (V*)**: Defined as the volume of the most succinct (concise) program in which a problem can be coded. This minimum volume is achieved when a program can be expressed using a single source code instruction, such as a simple function call.
* For an algorithm operating on `n` input/output data items (`d1, d2, ..., dn`), the most succinct program would be `f(d1, d2, ..., dn);` . In this case, `η1` (unique operators) would be 2 (for the function name `f` and the semicolon `;`), and `η2` (unique operands) would be `n` .
* Formula: `V* = (2 + η2) * log2(2 + η2)`
* **Program Level (L)**: Measures the level of abstraction provided by the programming language. A higher program level indicates that the language allows for more abstract expression, requiring less effort to develop a program.
* Formula: `L = V* / V`
* **Significance**: A higher `L` implies less effort is needed to develop a program in that language. This aligns with the observation that developing a program in a high-level language (like C) requires less effort than in assembly language (a low-level language).

7. Halstead's Metrics: Effort and Time

Halstead's Software Science also provides quantitative measures for the effort required to develop a program and the time a programmer would spend on it.
* **Programming Effort (E)**: Represents the number of mental discriminations required to implement, read, and understand the program.
* Initial Formula: `E = V / L` (Program Volume divided by Program Level).
* Substituting `L = V* / V` , the formula becomes: `E = V / (V* / V) = V^2 / V*`
* **Observation**: The programming effort `E` varies as the square of the program volume `V` .
* **Correlation**: Experience shows that `E` is well correlated with the effort needed for maintaining an existing program, reinforcing Halstead's utility for maintenance estimation.
* **Programmer's Time (T)**: The actual time a programmer would spend on the project.
* Formula: `T = E / S`
* `S` is the speed of mental discriminations, an empirically derived constant.

* **Recommended Value**: For programming applications, the recommended value for `S` is 18.

8. Strengths and Limitations

Halstead's Software Science offers a formal and quantitative approach to understanding software attributes.
* **Strengths**:
* Provides formal definitions and quantification for attributes like program complexity, ease of understanding, and level of abstraction.
* Based on low-level parameters (operators and operands), offering a granular analysis.
* Particularly effective for estimating software maintenance effort, where it often outperforms other estimation techniques.
* **Limitations**:
* While it provides gross estimates for large collections of software, its accuracy tends to decrease when applied to individual cases. For specific, isolated programs, the estimations might be less precise.

Key Takeaways

- [ ] Halstead's Software Science is an analytical estimation technique, distinct from empirical and heuristic methods due to its scientific basis.
- [ ] It is particularly effective for estimating software maintenance effort.
- [ ] The core parameters are the number of unique/total operators (`η1` , `N1`) and unique/total operands (`η2` , `N2`).
- [ ] Program length (`N = N1 + N2`) and program vocabulary (`η = η1 + η2`) are fundamental metrics.
- [ ] Program volume (`V = N * log2(η)`) compensates for programming language differences, representing the minimum bits to encode a program.
- [ ] Potential minimum volume (`V*`) and program level (`L = V* / V`) quantify inherent problem complexity and language abstraction.
- [ ] Programming effort (`E = V^2 / V*`) measures the mental discriminations required, correlating well with maintenance effort.
- [ ] Programmer's time (`T = E / S`) incorporates the speed of mental discriminations, with `S` typically set to 18.
- [ ] Halstead's theory provides gross estimates for large software collections but can be less accurate for individual cases.

Conclusion

Halstead's Software Science provides a rigorous, analytical framework for quantifying various attributes of a software project, moving beyond subjective guesses towards a more scientific understanding. By breaking down a program into its fundamental operators and operands, it allows for the calculation of metrics like length, vocabulary, volume, and crucially, the effort and time required for development and maintenance. This quantitative approach is invaluable for project managers seeking to make informed decisions, especially when planning for the long-term maintainability of software. While it excels in providing macro-level insights and is particularly strong for maintenance effort estimation, understanding its limitations for individual cases ensures its appropriate application. Ultimately, Halstead's Software Science equips practitioners with a powerful tool to better predict, plan, and manage the complexities inherent in software development lifecycles.